



ΗΥ340 : ΓΛΩΣΣΕΣ ΚΑΙ ΜΕΤΑΦΡΑΣΤΕΣ

ΠΑΝΕΠΙΣΤΗΜΙΟ ΚΡΗΤΗΣ,
ΣΧΟΛΗ ΘΕΤΙΚΩΝ ΕΠΙΣΤΗΜΩΝ,
ΤΜΗΜΑ ΕΠΙΣΤΗΜΗΣ ΥΠΟΛΟΓΙΣΤΩΝ

```
VAR i:Integer;  
    FUNCTION(Symbol) replicate  
        x = (function(x,y){return x+y;});  
        class DelFunctor: public std::unary_function<
```

ΔΙΔΑΣΚΩΝ

Αντώνιος Σαββίδης



HY340 : ΓΛΩΣΣΕΣ ΚΑΙ ΜΕΤΑΦΡΑΣΤΕΣ

Διάλεξη 2η ΛΕΞΙΚΟΓΡΑΦΙΚΗ ΑΝΑΛΥΣΗ - I



Περιεχόμενα

- *Ρόλος και αναγκαιότητα*
- Λεξικογραφικές δομές
- Κανονικές γλώσσες και εκφράσεις
- Διαγράμματα μετάβασης καταστάσεων
- Υλοποίηση διαγραμμάτων
- Μη αιτιοκρατικά αυτόματα και κανονικές εκφράσεις



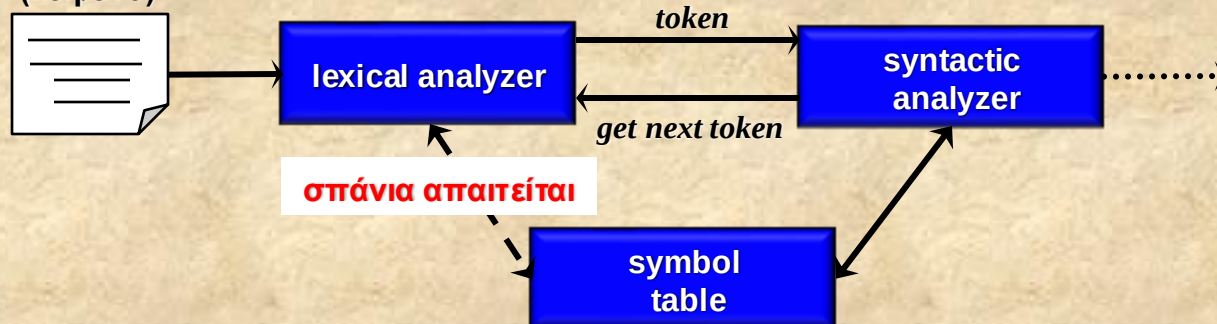
Ρόλος και αναγκαιότητα (1/2)

- Είναι η πρώτη φάση επεξεργασίας του αρχείου εισόδου, με σκοπό να παρέχει το επόμενο λεξικογραφικό στοιχείο (**token**) όποτε του «ζητηθεί»
 - Αυτή είναι και η μόνη αλληλεπίδραση που έχει με τα υπόλοιπα τμήματα του compiler
- Διαβάζει και μετατρέπει ακολουθίες χαρακτήρων ενός αρχείου εισόδου σε συγκεκριμένα λεξικογραφικά στοιχεία
 - στη διαδικασία αυτή πρέπει να αγνοεί κενά, νέες γραμμές, tabs και σχόλια
 - αλλά να «κρατάει» και τον εκάστοτε αριθμό των γραμμών κειμένου που έχουν ήδη διαβαστεί

Ρόλος και αναγκαιότητα (2/2)

- Η ύπαρξη ξεχωριστής λεξικογραφικής φάσης επιτρέπει
 - την απλούστευση της συντακτικής ανάλυσης
 - ◆ π.χ., η ενσωμάτωση της ανάλυσης των σχολίων και κενών στη συντακτική ανάλυση αυξάνει δραματικά την πολυπλοκότητα του συντακτικού αναλυτή
 - ενώ αυτά εύκολα μπορούν να αφαιρεθούν εύκολα από τον λεξικογραφικό αναλυτή
 - την διαφοροποίηση μεταξύ της λεξικογραφικής μορφής και γενικής κατηγορίας λεξικογραφικών στοιχείων
 - ◆ π.χ. μπορεί να κατασκευαστεί ένας λεξικογραφικός αναλυτής που αναγνωρίζει μορφές λέξεων-κλειδιών στην Ελληνική γλώσσα

πηγαίος κώδικας
(κείμενο)





Περιεχόμενα

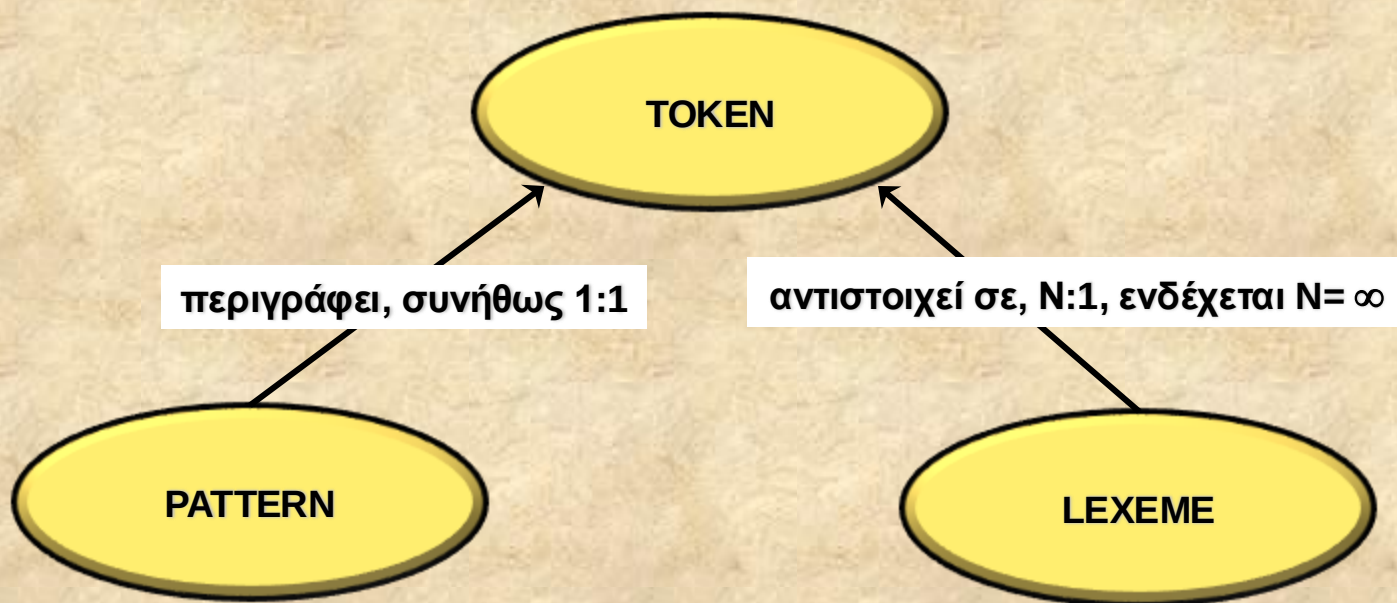
- Ρόλος και αναγκαιότητα
- *Λεξικογραφικές δομές*
- Κανονικές γλώσσες και εκφράσεις
- Διαγράμματα μετάβασης καταστάσεων
- Υλοποίηση διαγραμμάτων
- Μη αιτιοκρατικά αυτόματα και κανονικές εκφράσεις



Λεξικογραφικές δομές (1/6)

- Στη λεξικογραφική ανάλυση υπάρχει διαχωρισμός σε:
 - **Λεξικογραφικά στοιχεία – tokens**
 - ◆ οι βασικές κατηγορίες διακριτών στοιχείων, π.χ. αριθμοί, αναγνωριστικά, λέξεις κλειδιά, τελεστές, σημεία στίξης, κλπ
 - **Λέξεις – lexemes**
 - ◆ μία ακολουθία χαρακτήρων εισόδου η οποία αντιστοιχεί σε κάποιο συγκεκριμένο token, π.χ. τα 10, 20, 3.14 είναι όλα αριθμοί
 - **πρότυπα – patterns**
 - ◆ Η τυπική περιγραφή του συνόλου από λέξεις που αντιστοιχούν στο ίδιο λεξικογραφικό στοιχείο

Λεξικογραφικές δομές (2/6)



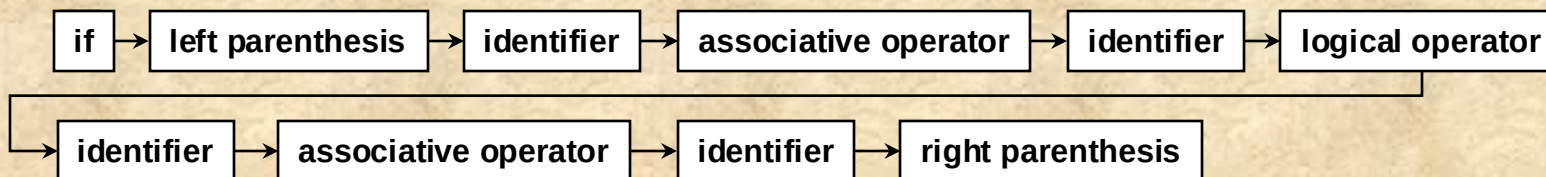


Λεξικογραφικές δομές (3/6)

Λεξικογραφικό στοιχείο	Παραδείγματα λέξεων	Άτυπη περιγραφή προτύπου
const if associative operator logical operator identifier integer constant real constant string literal left parenthesis right parenthesis	const if > >= < <= == != && ! x y count i totalItems 10 20 45 0 -34 3.14518 -346.67 "hello" "world" "<i,j>" ()	<i>const</i> <i>if</i> <i>> ή >= ή < ή <= ή == ή !=</i> <i>&& ή ή !</i> <i>πρώτα γράμμα, έπειτα γράμματα ή ψηφία</i> <i>ακέραια σταθερά</i> <i>πραγματική σταθερά</i> <i>χαρακτήρες μεταξύ " και " πλην του "</i> ()

if (x >= z || y <= x)

Η πρόταση αυτή αποτελείται από τα παρακάτω λεξικογραφικά στοιχεία:



Λεξικογραφικές δομές (4/6)

- Προφανώς πολλές λέξεις μπορεί να αναγνωρίζονται βάσει ενός λεξικογραφικού προτύπου
 - π.χ. οι λέξεις **10**, **35**, **-67** και **12** αναγνωρίζονται επιτυχώς να αντιστοιχούν όλες στο πρότυπο **ακέραιος αριθμός**
 - Ωστόσο, δεν αρκεί ο λεξικογραφικός αναλυτής απλώς να επιστρέφει την κατηγορία του προτύπου
 - ◆ **χρειάζεται να δίνει και πληροφορία σχετικά με το περιεχόμενο της εκάστοτε λέξης που είναι στιγμιότυπο του προτύπου**, π.χ.
 - μία λέξη που αντιστοιχεί σε ακέραιο αριθμό έχει την τιμή **-7**
 - μία λέξη που αντιστοιχεί σε αναγνωριστικό όνομα έχει την τιμή **Point**
 - μία λέξη που αντιστοιχεί σε σταθερά string έχει την τιμή **“Monday”**
- Έτσι εισάγουμε την έννοια των χαρακτηριστικών γνωρισμάτων για κάθε διαφορετική κατηγορία token
 - στα χαρακτηριστικά αυτά γνωρίσματα αποθηκεύεται η πληροφορία που περιέχει κάθε στοιχείο και η οποία υπολογίζεται κατά τη διάρκεια της λεξικογραφικής ανάλυσης

Λεξικογραφικές δομές (5/6)

- Μία απλή αλλά γενική μέθοδος υλοποίησης των token attributes είναι η παρακάτω:
 - κάθε token έχει ένα μόνο χαρακτηριστικό γνώρισμα
 - για τα αναγνωριστικά ονόματα έχουμε ένα κοινό τύπο χαρακτηριστικού γνωρίσματος, *symbol**, δηλ. δείκτη σε εγγραφές του symbol table
 - ◆ σε αυτή την εγγραφή αποθηκεύεται τόσο το όνομα, όσο και η επιπλέον πληροφορία που χρειάζεται για τα αναγνωριστικά
 - τα tokens που αντιστοιχούν σε σταθερές έχουν γνώρισμα με τύπο υλοποίησης ίδιο με αυτό της σταθεράς τιμής, π.χ. (C style):
 - ◆ `int val`, για σταθερά ακέραια τιμή,
 - ◆ `char* name`, για όνομα αναγνωριστικού (heap storage),
 - ◆ `char* val`, για σταθερά τιμή τύπου string (heap storage)

Λεξικογραφικές δομές (6/6)

- Ακολουθεί παράδειγμα που δείχνει τα παραγόμενα tokens και τα γνωρίσματά τους για μία εντολή

x = f (2 * y + 2 * z);

symbol table

x
y
f
z

<id, **sym** δείκτης στον symbol table>
 <assign, >
 <id, **sym** δείκτης στον symbol table>
 <left parenthesis, >
 <integer const, val = 2>
 <mul, >
 <id, **sym** δείκτης στον symbol table>
 <add, >
 <integer const, val = 2>
 <mul, >
 <id, **sym** δείκτης στον symbol table>
 <right parenthesis,>
 <semicolon,>

Η καταχώρηση σε symbol table δε συνηθίζεται στη λεξικογραφική ανάλυση – είναι απλώς παράδειγμα



Περιεχόμενα

- Ρόλος και αναγκαιότητα
- Λεξικογραφικές δομές
- *Κανονικές γλώσσες και εκφράσεις*
- Διαγράμματα μετάβασης καταστάσεων
- Υλοποίηση διαγραμμάτων
- Μη αιτιοκρατικά αυτόματα και κανονικές εκφράσεις



Κανονικές γλώσσες και εκφράσεις (1/5)

- Ένας πολύ βολικός τρόπος περιγραφής των λεξικογραφικών προτύπων είναι οι κανονικές εκφράσεις
 - Κάθε διαφορετικό λεξικογραφικό πρότυπο αντιστοιχεί σε μία κανονική γλώσσα
 - Αυτή περιλαμβάνει όλες τις λέξεις που ικανοποιούν το λεξικογραφικό πρότυπο
 - ◆ Πχ., για το πρότυπο **identifier** είχαμε δώσει τον άτυπο ορισμό «**πρώτα γράμμα, έπειτα γράμματα η ψηφία**», που αντιστοιχεί στην κανονική έκφραση
 - ***letter (letter| digit) ****
- ➔ *Ακολουθεί μία υπενθύμιση των πιο σημαντικών ορισμών καθώς θα μας φανούν χρήσιμοι*



Κανονικές γλώσσες και εκφράσεις (2/5)

Εάν Σ είναι κάποιο αλφάβητο, οι κανονικές εκφράσεις που προέρχονται από το Σ , καθώς και οι αντίστοιχες κανονικές γλώσσες ορίζονται βάσει των κανόνων:

1. Το $\emptyset$ είναι κανονική έκφραση και παριστάνει την κενή κανονική γλώσσα.
2. Το ε είναι μία κανονική έκφραση και παριστάνει την κανονική γλώσσα $\{\varepsilon\}$ που περιέχει την κενή λέξη.
3. $\forall a \in \Sigma$, το a είναι κανονική έκφραση και αναπαριστά την κανονική γλώσσα $\{a\}$.
4. Αν r και s είναι κανονικές εκφράσεις για τις κανονικές γλώσσες $L(r)$ και $L(s)$, τότε οι εκφράσεις (r) , (s) , $r \mid s$, $r s$ και r^* είναι και αυτές κανονικές, για τις αντίστοιχες κανονικές γλώσσες $L(r)$, $L(r) \cup L(s)$, $L(r)L(s)$ και $(L(r))^*$.



Κανονικές γλώσσες και εκφράσεις (3/5)

Παρακάτω ακολουθούν ορισμοί των βασικών αλγεβρικών ιδιοτήτων των κανονικών εκφράσεων

1. $r | s = s | r$, δηλ. $|$ είναι αντιμεταθετική
2. $r | (s | t) = (r | s) | t$, δηλ. $|$ είναι προσεταιριστική
3. $(rs)t = (rs)t$, δηλ. η συνένωση είναι προσεταιριστική
4. $r(s | t) = rs | rt$ και $(s | t)r = sr | tr$, η συνένωση είναι επιμεριστική ως προς την $|$
5. $\epsilon r = r$ και $r\epsilon = r$, δηλ. η κενή λέξη είναι το ουδέτερο στοιχείο της συνένωσης
6. $r^* = (r | \epsilon)^*$, η σχέση μεταξύ $*$ και ϵ
7. $r^{**} = r^*$, δηλ. η $*$ είναι αυτοπαθής.

Κανονικές γλώσσες και εκφράσεις (4/5)

Αν Σ είναι ένα αλφάβητο, τότε ένας κανονικός ορισμός είναι μία ακολουθία της μορφής:

$d_1 \rightarrow r_1$	}	<code>#define d_j r_j</code> Δηλαδή είναι κάτι αντίστοιχο με τα macros, για να μη ξαναγράψουμε συνέχεια τις εκφράσεις
$d_2 \rightarrow r_2$		
$\dots$ $d_n \rightarrow r_n$		

όπου κάθε d_i είναι ένα ξεχωριστό όνομα και κάθε r_i είναι μία κανονική έκφραση με σύμβολα από το $\Sigma \cup \{d_1, d_2, \dots, d_{i-1}\}$

Εάν r είναι κανονική έκφραση τότε:

r^+ είναι συντομογραφία για την κανονική έκφραση rr^*

$r?$ είναι συντομογραφία για την κανονική έκφραση $r \mid \varepsilon$



Κανονικές γλώσσες και εκφράσεις (5/5)

Παράδειγμα

letter → A | B | ... | Z | a | b | ... | z

digit → 0 | 1 | ... | 9

identifier → letter (letter | digit)*

intnum → digit+

realnum → intnum . digit+

delimiter → blank | tab | newline

whitespace → delimiter+

- Στο παραπάνω παράδειγμα δίνονται οι κανονικοί ορισμοί για τα λεξικογραφικά πρότυπα των εξής tokens:
 - **identifier** - αναγνωριστικό όνομα
 - **intnum** - σταθερά ακέραιος αριθμός
 - **realnum** - σταθερά πραγματικός αριθμός
 - **whitespace** - κενός χώρος μεταξύ των λέξεων



Περιεχόμενα

- Ρόλος και αναγκαιότητα
- Λεξικογραφικές δομές
- Κανονικές γλώσσες και εκφράσεις
- *Διαγράμματα μετάβασης καταστάσεων*
- Υλοποίηση διαγραμμάτων
- Μη αιτιοκρατικά αυτόματα και κανονικές εκφράσεις



Διαγράμματα μετάβασης καταστάσεων (1/4)

- Καθώς έχουμε επιλέξει τις κανονικές εκφράσεις ως μία κατάλληλη μέθοδο για την περιγραφή των λεξικογραφικών προτύπων, το επόμενο βήμα στην κατασκευή ενός αναλυτή είναι
 - **η εύρεση ενός αλγοριθμικού τρόπου** αναγνώρισης των προτύπων
 - καθώς και η κατάλληλη **μετουσίωσή του τρόπου αυτού σε κώδικα**
- Εφόσον έχουμε κανονικές εκφράσεις, ένας απλός τρόπος σχεδίασης «μηχανιστικών τρόπων» αναγνώρισης είναι τα διαγράμματα μετάβασης καταστάσεων – **state transition diagrams**
 - Τα διαγράμματα αυτά είναι εύκολα τόσο σχεδίαση, στην ανάγνωση και κατανόηση (από τρίτους), αλλά και στην υλοποίηση (κώδικας)

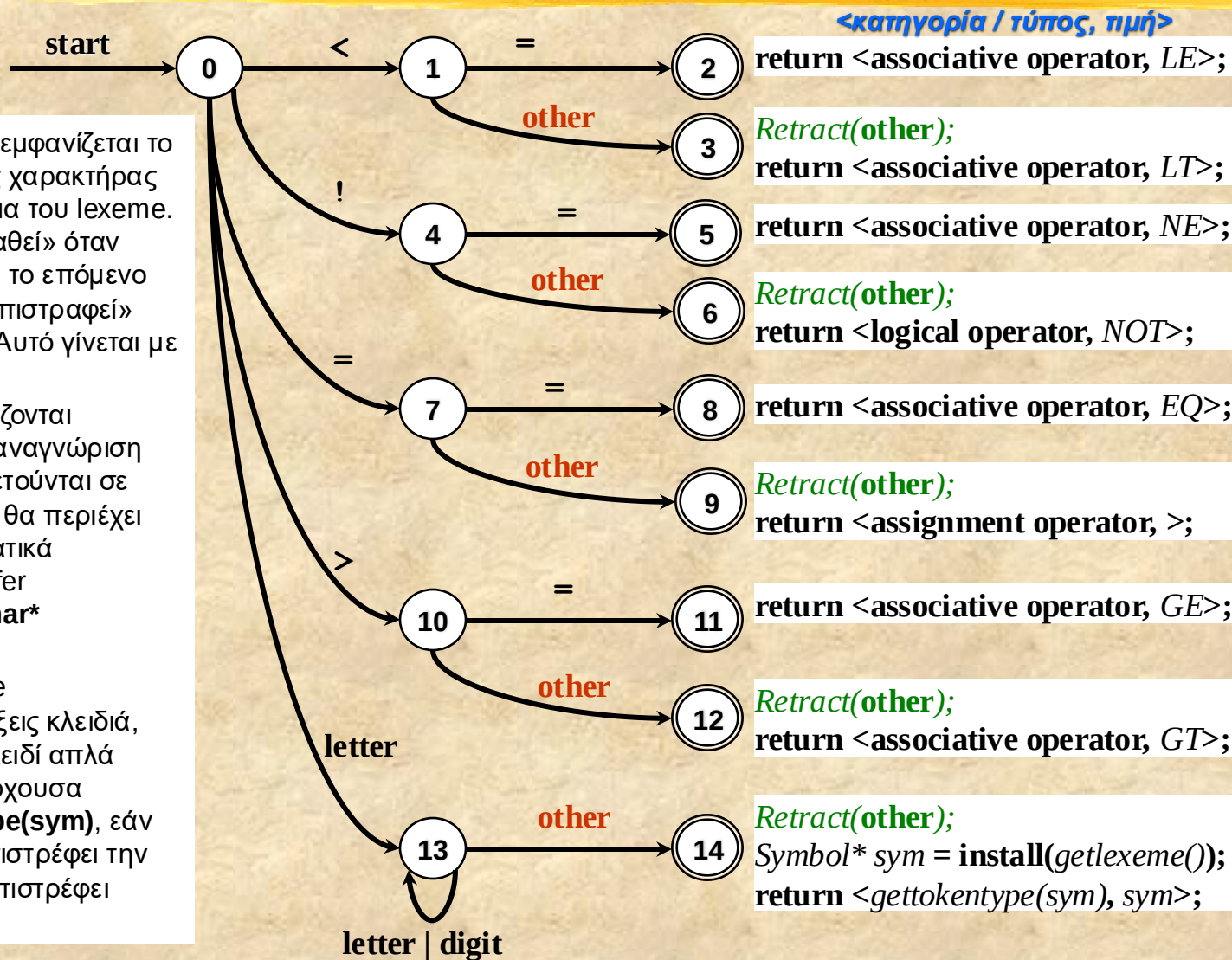


Διαγράμματα μετάβασης καταστάσεων (2/4)

1. Στις μεταβάσεις όπου εμφανίζεται το **other**, ουσιαστικά όποιος χαρακτήρας διαβάζεται δεν είναι τμήμα του lexeme. Όμως πρέπει να μην «χαθεί» όταν πάμε να αναγνωρίσουμε το επόμενο token, άρα πρέπει να «επιστραφεί» στον όποιο Input buffer. Αυτό γίνεται με την **Retract(char c)**.

2. Κάθε φορά που διαβάζονται χαρακτήρες για να γίνει αναγνώριση ενός token, αυτοί τοποθετούνται σε ένα char buffer ο οποίος θα περιέχει και την λέξη που πραγματικά διαβάστηκε. Αυτός ο buffer επιστρέφεται από την **char* getlexeme()**.

3. Εάν στον symbol table αποθηκεύουμε και τις λέξεις κλειδιά, τότε η **install** για λέξη κλειδί απλά επιστρέφει την ήδη υπάρχουσα εγγραφή. Η **gettokentype(sym)**, εάν **sym** είναι λέξη κλειδί, επιστρέφει την κατηγορία του, αλλιώς επιστρέφει **identifier**.





Διαγράμματα μετάβασης καταστάσεων (3/4)

- Ο lexical analyzer πρέπει να έχει τη δυνατότητα να «επιστρέφει» στην είσοδο ένα χαρακτήρα που ήδη διάβασε - **retract**, για τις μεταβάσεις που εμφανίζεται το *other* – αυτό λέγεται **look ahead** (κοιτάω μπροστά)
- Επειδή οι λέξεις κλειδιά προκαλούν την εισαγωγή ενός μεγάλου αριθμού από καταστάσεις (εκατοντάδες), δεν είναι καθόλου βολικό να τις αντιμετωπίζουμε ξεχωριστά, αλλά:
 - Τις αντιμετωπίζουμε ως identifier tokens, τα οποία πριν την έναρξη της λεξικογραφικής ανάλυσης τα κάνουμε install στον symbol table, με το flag ότι είναι *keywords* και με την ειδική κατηγορία τους (π.χ. IF, THEN, WHILE, κλπ).
 - Όταν αναγνωριστεί ένα identifier, τότε ελέγχουμε εάν υπάρχει ήδη στον symbol table:
 - Εάν ναι και είναι *id*, τότε το token που αναγνωρίστηκε είναι ένα identifier που ήδη έχει καταχωρηθεί πριν, αλλιώς
 - Εάν ναι και είναι *keyword*, τότε το token που αναγνωρίστηκε είναι η συγκεκριμένη κατηγορία keyword, αλλιώς
 - Αναγνωρίστηκε ένα νέο id το οποίο και καταχωρείται.



Περιεχόμενα

- Ρόλος και αναγκαιότητα
- Λεξικογραφικές δομές
- Κανονικές γλώσσες και εκφράσεις
- Διαγράμματα μετάβασης καταστάσεων
- *Υλοποίηση διαγραμμάτων*
- Μη αιτιοκρατικά αυτόματα και κανονικές εκφράσεις



Υλοποίηση διαγραμμάτων (1/7)

Διάβασμα αρχείου για λεξικογραφική ανάλυση με *look ahead* ενός χαρακτήρα (LA=1)

```
#include <stdio.h>
#include <assert.h>
#include <ctype.h>

#define MAX_LEXEME 1024

char    lookAhead = '\0';
int     useLookAhead = 0;
FILE*   inputFile = (FILE*) 0;
char    lexeme[MAX_LEXEME];
unsigned curr = 0;
unsigned lineNo = 0;

void ResetLexeme (void)
{ curr = 0; }

char* GetLexeme (void) {
    lexeme[curr] = '\0';
    return lexeme;
}
```

```
char GetNextChar (void) {
    if (useLookAhead) {
        useLookAhead = 0;
        return lookAhead;
    }
    else {
        assert(!feof(inputFile));
        return fgetc(inputFile);
    }
}

void Retract (char c) {
    assert(!useLookAhead);
    useLookAhead = 1;
    lookAhead = c;
}

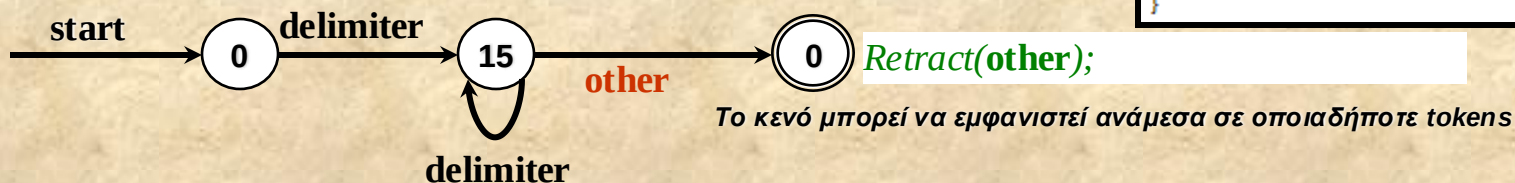
void CheckLine (char c)
{ if (c == '\n') ++lineNo; }
```

Επειδή έχουμε LA=1

lexeme

h	e	l	l	o	.	.	.	.	.	.	.	.	.	.
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

```
void ExtendLexeme (char c) {
    assert(curr < MAX_LEXEME - 1);
    lexeme[curr++] = c;
}
```





Υλοποίηση διαγραμμάτων (2/7)

Ξεχωριστά macros για κάθε διαφορετική κατηγορία token (και enumerated κάνουν...)

Από αυτή την ανακύκλωση «βγαίνουμε» για τρεις λόγους: (α) αναγνώριση token, (β) τέλος αρχείου, (γ) άγνωστο token (error).

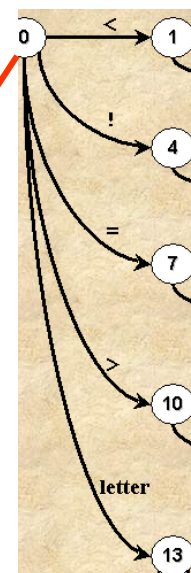
Εδώ είναι ένα βασικό case statement, το οποίο μας οδηγεί από το start state (0) στο κατάλληλο επόμενο state (δηλ. το αρχικό state του υπό-διαγράμματος)

```
#define UNKNOWN_TOKEN -1
#define END_OF_FILE 0
#define LE 1
#define LT 2
#define NE 3
#define NOT 4
#define IDENTIFIER 5
```

```
int state = 0;
unsigned gettoken (void) {
    state = 0;
    ResetLexeme();
    while (1) {
        if (state == -1)
            return UNKNOWN_TOKEN;
        if (feof(inputFile))
            return END_OF_FILE;
        char c = GetNextChar();
```

```
        switch (state) {
            case 0 :
                if (c == '<') state = 1; else
                if (c == '!') state = 4; else
                if (c == '=') state = 7; else
                if (c == '>') state = 10; else
                if (isalpha(c)) state = 13; else
                if (isspace(c))
                    { CheckLine(c); state = 15; continue; }
                else
                    { state = -1; continue; }
                break;
```

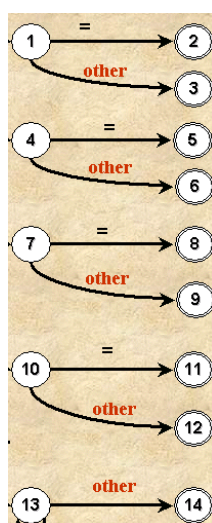
State hard-coding (1/2)



Υλοποίηση διαγραμμάτων (3/7)

State hard-coding (2/2)

Εδώ είναι όλες οι ξεχωριστές περιπτώσεις (cases) για κάθε state. Το if είναι ο έλεγχος για σύμβολο (τα βελάκια του διαγράμματος) ενώ η υλοποίηση της case εντολής υλοποιεί την λογική μετάβασης (νέο state ή αναγνώριση του token)



```
case 1:
    if (c == '=') { ExtendLexeme(c); return LE; }
    Retract(c); return LT;
case 4:
    if (c == '=') { ExtendLexeme(c); return NE; }
    Retract(c); return NOT;
case 15:
    if (isspace(c))
        { CheckLine(c); state = 15; }
    else
        { Retract(c); state=0; }
    continue;
case 13:
    if (isalpha(c) || isdigit(c)) state = 13;
    else { Retract(c); return IDENTIFIER; }
    break;

default: assert(0); /* How did we get here? */
}
ExtendLexeme(c);
}
```



space

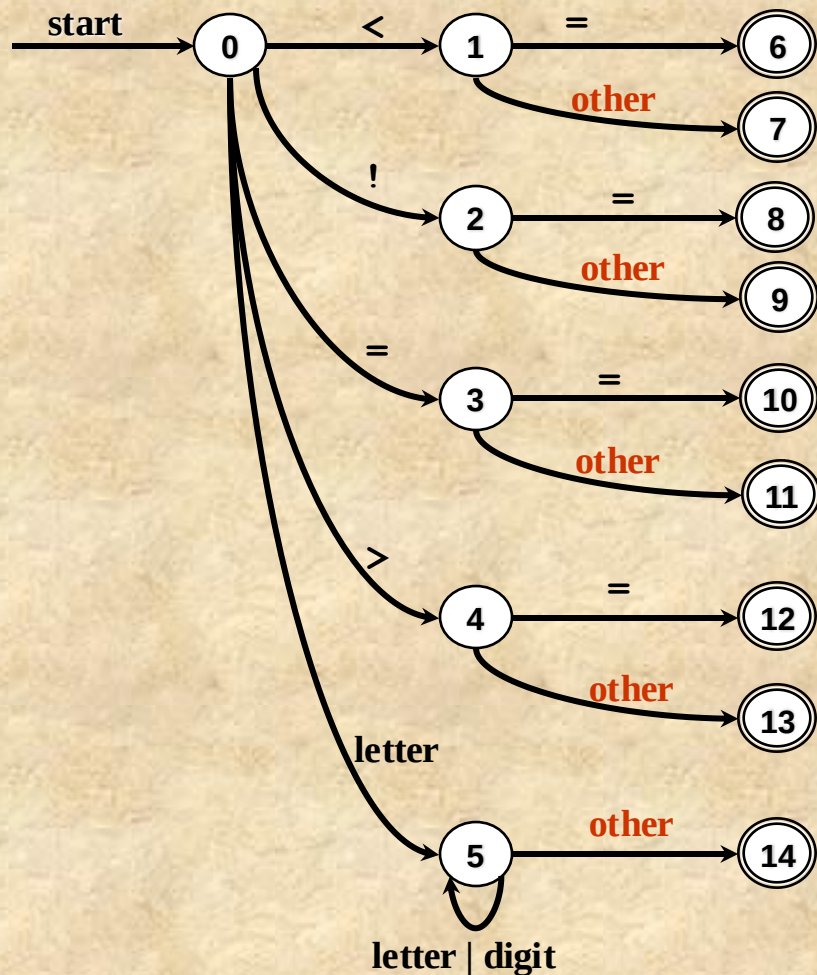


letter | digit

Επίσης φαίνονται και οι δύο περιπτώσεις (με κόκκινα περιγράμματα) όπου η μετάβαση ουσιαστικά παραμένει στην ίδια κατάσταση, το οποίο στις κανονικές εκφράσεις ισοδυναμεί με το *. Προφανώς η εκχώρηση είναι περιττή, αλλά τεκμηριώνει καλύτερα τι συμβαίνει.

Στη συγκεκριμένη υλοποίηση της *gettoken* επιστρέφεται ένας μοναδικός αριθμός που χαρακτηρίζει την κατηγορία του εκάστοτε αναγνωρισμένου token. Η βασική *GetToken* μπορεί τώρα να υλοποιηθεί πολύ εύκολα ώστε, καλώντας εσωτερικά την *gettoken*, να επιστρέφει token type και attribute, βάσει του προηγούμενου διαγράμματος.

Υλοποίηση διαγραμμάτων (4/7)



State transition tables (1/3)

• Έχουμε ήδη παρατηρήσει ότι οι τελικές καταστάσεις δεν χρησιμοποιούνται στον κώδικα, αλλά μόνο μέσα στο διάγραμμα.

• Καθώς η αρίθμηση των καταστάσεων είναι ουσιαστικά αυθαίρετη και γίνεται για δική μας ευκολία, τροποποιούμε το προηγούμενο διάγραμμα ώστε οι αρχικές καταστάσεις να λαμβάνουν συνεχόμενους αριθμούς.

• Εάν κάνουμε κάτι τέτοιο, τα case statements θα έχουν συνεχόμενους αριθμούς.

• Το παραπάνω μας κάνει να σκεφτούμε τη δυνατότητα να μετατρέψουμε τον κώδικα κάθε case σε συνάρτηση

• Έπειτα αποθηκεύουμε τη διεύθυνση κάθε τέτοιας συνάρτησης σε ένα πίνακα σε θέση ίδια με την τιμή του αντίστοιχου state που εμφανίζεται στο case.

Υλοποίηση διαγραμμάτων (5/7)

State transition tables (2/3)

```
typedef int (*StateFunc) (char c);

#define MAX_STATE      6
#define TOKEN_SHIFT    (MAX_STATE+1)
#define TOKEN(t)       TOKEN_SHIFT+(t)
#define STATE(s)       s
#define istoken(s)     ((s)>MAX_STATE)

int sf0 (char c) {
    if (c == '<') return STATE(1);
    if (c == '!') return STATE(2);
    if (c == '=') return STATE(3);
    if (c == '>') return STATE(4);
    if (isalpha(c)) return STATE(5);
    if (isspace(c))
        { CheckLine(c); return STATE(6); }
    return STATE(-1);
}
```

```
int sf1 (char c) {
    if (c == '=') { ExtendLexeme(c); return TOKEN(LE); }
    Retract(c); return TOKEN(LT);
}

int sf2 (char c) {
    if (c == '=') { ExtendLexeme(c); return TOKEN(NE); }
    Retract(c); return TOKEN(NOT);
}
```

•Θα χρησιμοποιήσουμε έναν πίνακα όπου για κάθε κατάσταση θα έχει μία συνάρτηση η οποία υλοποιεί τη λογική μετάβασης - αυτή που είχαμε πριν σε ξεχωριστά case statements.

•Προφανώς χρειαζόμαστε έναν πίνακα μεγέθους όσα και τα states από συναρτήσεις. Ο τύπος της συνάρτησης που επιλέγουμε, προφανώς έχει ως παράμετρο τον χαρακτήρα προς επεξεργασία, ενώ επιστρέφει έναν αριθμό 'n' για τον οποίο ισχύει:

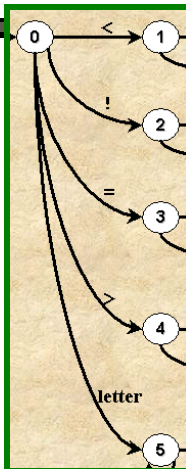
- Αν $n > MAX_STATE$, τότε μόλις αναγνωρίστηκε το token με αριθμό $n - MAX_STATE - 1$
- Αλλιώς έχουμε μετάβαση στη νέα κατάσταση state n.

```
int sf3 (char c);
int sf4 (char c);

int sf5 (char c) {
    if (isalpha(c) || isdigit(c))
        return STATE(5);
    Retract(c);
    return TOKEN(IDENTIFIER);
}
```

```
int sf6 (char c) {
    if (isspace(c))
        { CheckLine(c); return STATE(6); }
    Retract(c); return STATE(0);
}
```

```
StateFunc stateFuncs[MAX_STATE+1]={
    &sf0, &sf1, &sf2, &sf3, &sf4, &sf5, &sf6
};
```





Υλοποίηση διαγραμμάτων (6/7)

- Αυτή η τεχνική κατασκευής έχει το καλό ότι επιτρέπει η *gettoken2* να υφίσταται όντας ανεξάρτητη τόσο του αριθμού των καταστάσεων, όσο και της λογικής μετάβασης, δηλ. παραμένει σταθερή.

- Επιπλέον, η λογική μετάβασης κωδικοποιείται σε ξεχωριστές συναρτήσεις, κάτι που προωθεί την πιο εύκολη διαχείριση αλλά και συντήρηση του κώδικα.

- Οι όποιες προσθήκες επιπλέον καταστάσεων και συναρτήσεων μετάβασης είναι εφικτές εύκολα, χωρίς να επηρεάζονται άλλες συναρτήσεις.

Η λογική μετάβασης κατάστασης ανάλογα με τον χαρακτήρα εισόδου.

Πως αναγνωρίζουμε εάν νέο token έχει επιτυχώς αναγνωρισθεί.

State transition tables (3/3)

```
unsigned gettoken2 (void) {
    state = 0;
    ResetLexeme();
    while (1) {
        if (feof(inputFile))
            return END_OF_FILE;

        char c = GetNextChar();

        state = (*stateFuncs[state])(c);

        if (state == -1)
            return UNKNOWN_TOKEN;
        else
            if (istoken(state))
                return state - TOKEN_SHIFT;
            else
                ExtendLexeme(c);
    }
}
```



Υλοποίηση διαγραμμάτων (7/7)

```
unsigned gettoken3 (void) {
    while (1) {
        if (feof(inputFile))
            return END_OF_FILE;
        char c = GetNextChar();

        if (isspace(c)) {
            if (c=='\n') ++lineNo;
        }
        else
            if (c == '<') {
                if ((c = GetNextChar()) == '=')
                    return LE;
                else { Retract(c); return LT; }
            }
            else
                if (c == '>') {
                    if ((c = GetNextChar()) == '=')
                        return GE;
                    else { Retract(c); return GT; }
                }
    }
}
```

• Φαίνεται ότι οι μεταβάσεις του ίδιου υπό-διαγράμματος οδηγούν συνήθως σε φωλιασμένες if / else εντολές.
• Η υλοποίηση των εκφράσεων που ορίζονται με * οδηγεί σε φωλιασμένη ανακύκλωση. Προσέξτε ότι το white space δεν αντιμετωπίζεται ως delimiter*, αλλά απλά ως delimiter που «απορροφάται» (δεν επιστρέφει token, απλώς ελέγχεται για αλλαγή γραμμής).

Logic hard-coding (1/1)

```
if (c == '!') {
    if ((c = GetNextChar()) == '=')
        return NE;
    else { Retract(c); return NOT; }
}
else
    if (c == '=') {
        if ((c = GetNextChar()) == '=')
            return EQ;
        else { Retract(c); return ASSIGN; }
    }
    else
        if (isalpha(c)) {
            do {
                lexeme[curr++] = c;
                c = GetNextChar();
            } while (isalpha(c) || isdigit(c));
            Retract(c);
            return IDENTIFIER;
        }
        else
            return UNKNOWN_TOKEN;
    }
}
```



Περιεχόμενα

- Ρόλος και αναγκαιότητα
- Λεξικογραφικές δομές
- Κανονικές γλώσσες και εκφράσεις
- Διαγράμματα μετάβασης καταστάσεων
- Υλοποίηση διαγραμμάτων
- *Μη αιτιοκρατικά αυτόματα και κανονικές εκφράσεις*

Μη αιτιοκρατικά αυτόματα (1/7)

- Μελετώντας τις προηγούμενες τεχνικές υλοποίησης συναρτήσεων για αναγνώρισης προτύπων των tokens
 - βάσει των διαγραμμάτων μετάβασης καταστάσεως
 - τα οποία αντιστοιχούσαν στις κανονικές εκφράσεις των προτύπων
- γίνεται πολύ γρήγορα αντιληπτό ένα επαναλαμβανόμενο μοτίβο στον κώδικα
 - το οποίο μάλιστα θα μπορούσε αντί κάποιος να το γράφει ξαν'α και ξανά κάθε φορά
 - να σχεδιάσει έναν αλγόριθμο που θα μπορούσε όταν εκτελείται να παράγει τον κώδικα του λεξικογραφικού αναλυτή αυτόματα
- Θεωρητικά, οι προηγούμενες οδηγίες κατασκευής σκιαγραφούν τέτοιους αλγόριθμους «ημιαυτόματης» παραγωγής κώδικα λεξικογραφικού αναλυτή
 - Απαιτείται όμως η σχεδίαση του διαγράμματος μετάβασης καταστάσεων η οποία δεν έχει αυτοματοποιηθεί



Μη αιτιοκρατικά αυτόματα (2/7)

■ Ο στόχος

- Από μία κανονική έκφραση στην αυτόματη παραγωγή ενός λεξικογραφικού αναλυτή

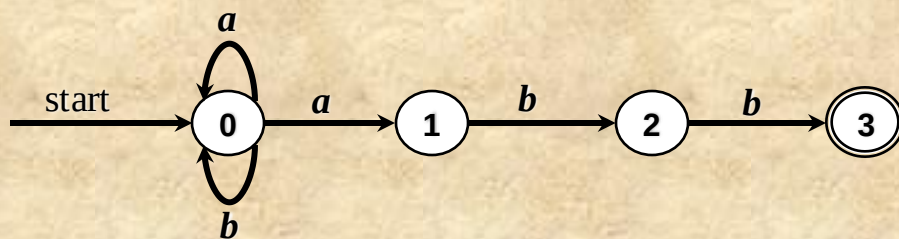
■ Διαδοχικά βήματα αυτοματοποίησης (με αλγορίθμους)

- Κανονική έκφραση $\rightarrow$ Μη αιτιοκρατικό αυτόματο
 - ◆ Απλά γιατί είναι ευκολότερο από ότι για αιτιοκρατικά
- Μη αιτιοκρατικό αυτόματο $\rightarrow$ αιτιοκρατικό αυτόματο
 - ◆ Σχετικά πολύπλοκος αλγόριθμος
- Αιτιοκρατικό αυτόματο $\rightarrow$ εξομοίωση λειτουργίας
 - ◆ Απλός παγιωμένος αλγόριθμος

Μη αιτιοκρατικά αυτόματα (3/7)

Ένα μη αιτιοκρατικό αυτόματο (Non-deterministic finite automaton – **NFA**) είναι ένα μαθηματικό μοντέλο που περιέχει:

1. Ένα σύνολο από καταστάσεις **S**
2. Ένα αλφάβητο **Σ** συμβόλων εισόδου
3. Μία συνάρτηση μετάβασης **move** η οποία αντιστοιχεί ζεύγη κατάστασης και συμβόλου εισόδου σε υποσύνολο του **S**
4. Μία κατάσταση **s_0** η οποία διακρίνεται ως η αρχική κατάσταση
5. Ένα υποσύνολο καταστάσεων **F** του **S** που διακρίνονται ως τελικές



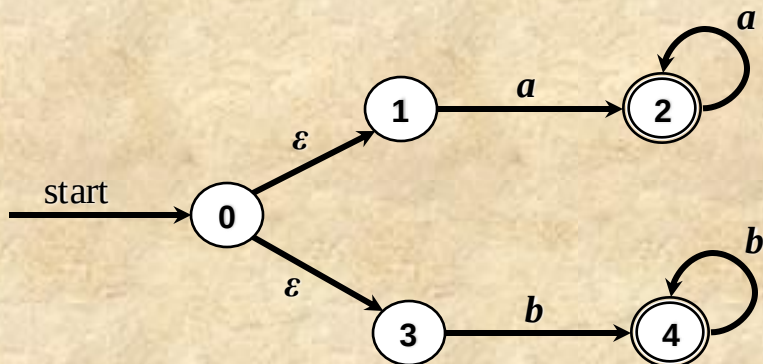
Κατάσταση	Σύμβολο εισόδου	
	<i>a</i>	<i>b</i>
0	{0,1}	{0}
1	-	{2}
2	-	{3}

NFA για την αναγνώριση της γλώσσας $(a \mid b)^*abb$ και ο αντίστοιχος πίνακας μετάβασης καταστάσεων

Μη αιτιοκρατικά αυτόματα (4/7)

Παραγωγή / αναγνώριση λέξεων

- Οι κανονικές εκφράσεις παρέχουν έναν τρόπο παραγωγής όλων των λέξεων μίας γλώσσας (κανονικής).
- Τα αυτόματα παρέχουν έναν τρόπο αναγνώρισης εάν κάποια συγκεκριμένη λέξη ανήκει σε κάποια συγκεκριμένη κανονική γλώσσα.
- Μεταξύ τους έχουν διαφορετικές φιλοσοφίες, ωστόσο θεωρητικά είναι ισοδύναμα (οι κανονικές εκφράσεις και τα αυτόματα).
- Η δέουσα πρακτική
 - Χρησιμοποιούμε κανονικές εκφράσεις για τον ορισμό της γλώσσας
 - Αυτές μεταφράζονται αυτομάτως σε πεπερασμένα αυτόματα για την υλοποίηση



NFA για την αναγνώριση της γλώσσας $(aa^*)|(bb^*)$

Κανονική έκφραση $\rightarrow$ NFA

- Η κατασκευή θα γίνει επαγωγικά ως προς τη δομή, με σχετικά απλό τρόπο
- Δεδομένης μίας οποιασδήποτε κανονικής έκφρασης r ο αλγόριθμος βασίζεται στη συντακτική δομή της r :
 - Πρώτα φτιάχνουμε αυτόματα που αναγνωρίζουν το ϵ καθώς και όλα τα σύμβολα του αλφαβήτου
 - Έπειτα φτιάχνουμε αυτόματα για εκφράσεις που περιέχουν $|$ ή **συνένωση** ή το $*$

Μη αιτιοκρατικά αυτόματα (5/7)

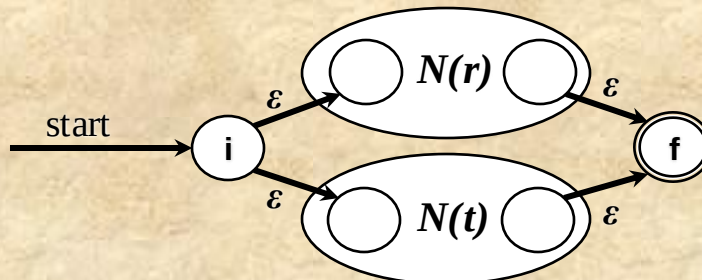
Για το ϵ δημιουργούμε το NFA, με **i (initial)** η νέα αρχική κατάσταση και **f (final)** τη νέα τελική κατάσταση. Το αυτόματο αυτό αναγνωρίζει την $\{\epsilon\}$



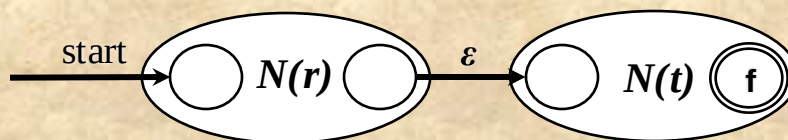
Για κάθε εμφάνιση του a από το Σ στην έκφραση δημιουργούμε το NFA με **i** η νέα αρχική κατάσταση και **f** τη νέα τελική κατάσταση. Το αυτόματο αυτό αναγνωρίζει την $\{a\}$



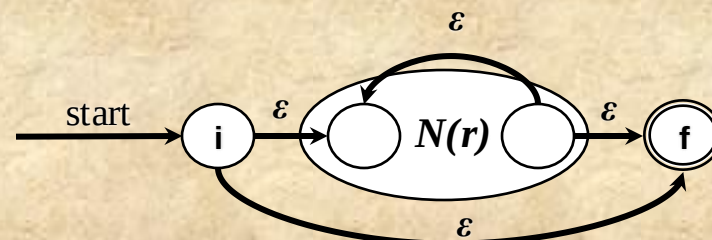
Για τις κανονικές εκφράσεις r και t με αντίστοιχα NFA $N(r)$ και $N(t)$, δημιουργούμε το NFA $(r|t)$, με **i** η νέα αρχική κατάσταση και **f** τη νέα τελική κατάσταση. **Μεταβάσεις από το i στις αρχικές καταστάσεις των $N(r)$ & $N(t)$ και προς το f από τις τελικές τους.** Το αυτόματο αυτό αναγνωρίζει την $L(r) \cup L(t)$



Για τις κανονικές εκφράσεις r και t με αντίστοιχα NFA $N(r)$ και $N(t)$, δημιουργούμε το NFA (rt) , με **αρχική κατάσταση την αρχική του $N(r)$ και τελική κατάσταση την τελική του $N(t)$ και μεταβάσεις από τις τελικές $N(r)$ στις αρχικές του $N(t)$.** Το αυτόματο αυτό αναγνωρίζει την $L(r) L(t)$

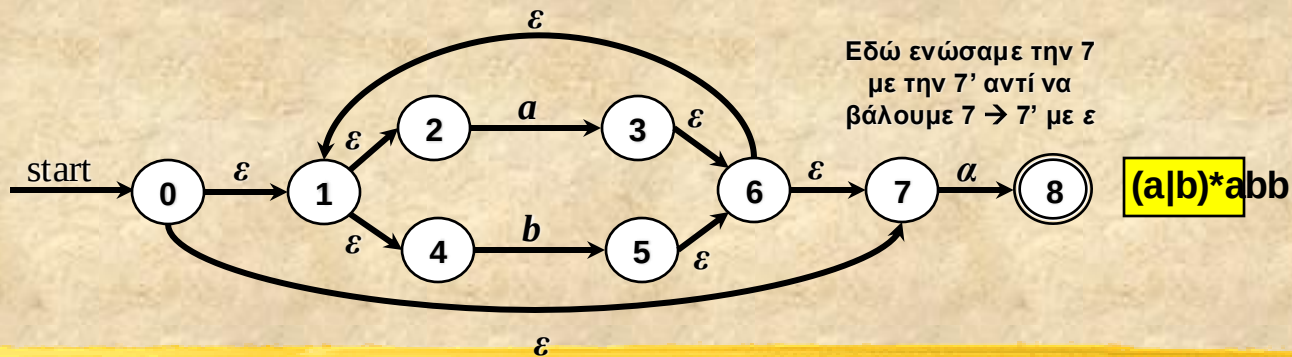
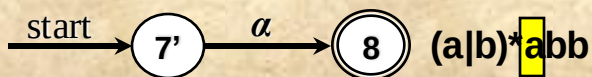
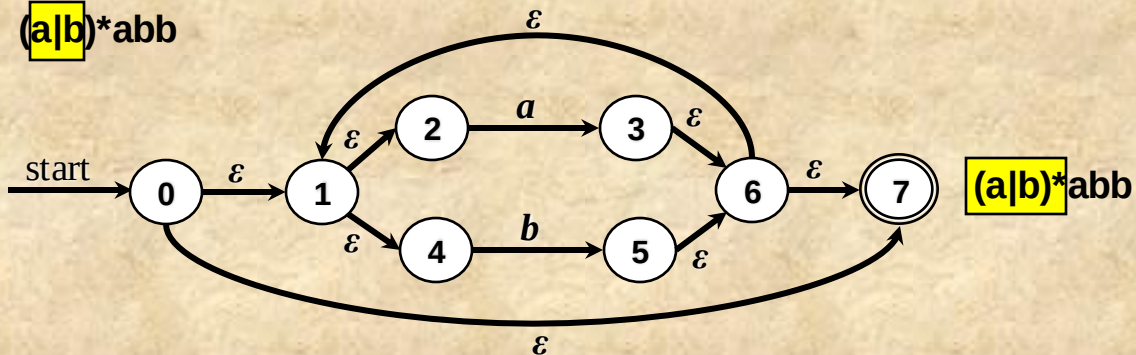
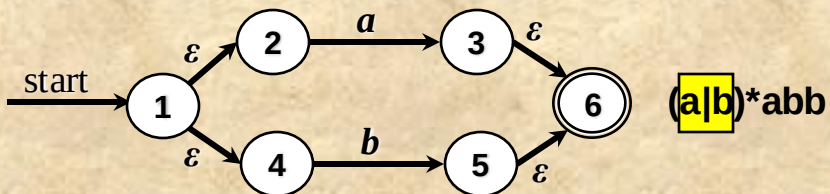
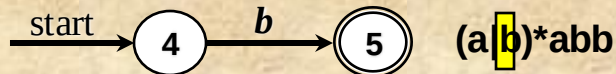
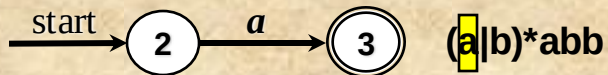


Για την κανονική έκφραση r^* δημιουργούμε το NFA (r^*) , με **i** η νέα αρχική κατάσταση και **f** τη νέα τελική κατάσταση. Το αυτόματο αυτό αναγνωρίζει την $(L(r))^*$



Μη αιτιοκρατικά αυτόματα (6/7)

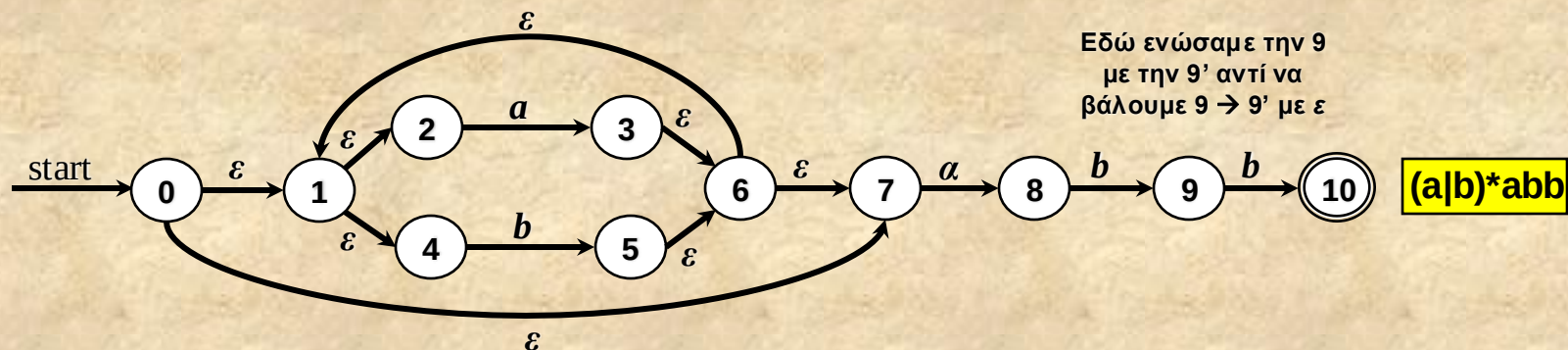
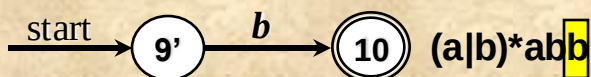
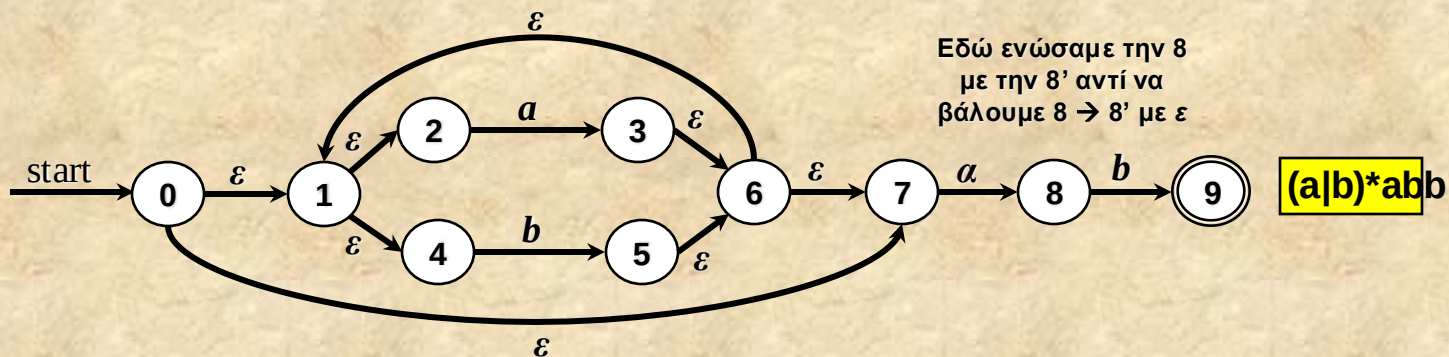
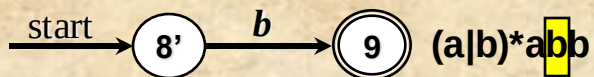
Παράδειγμα. Κατασκευή NFA για την έκφραση $(a|b)^*abb$



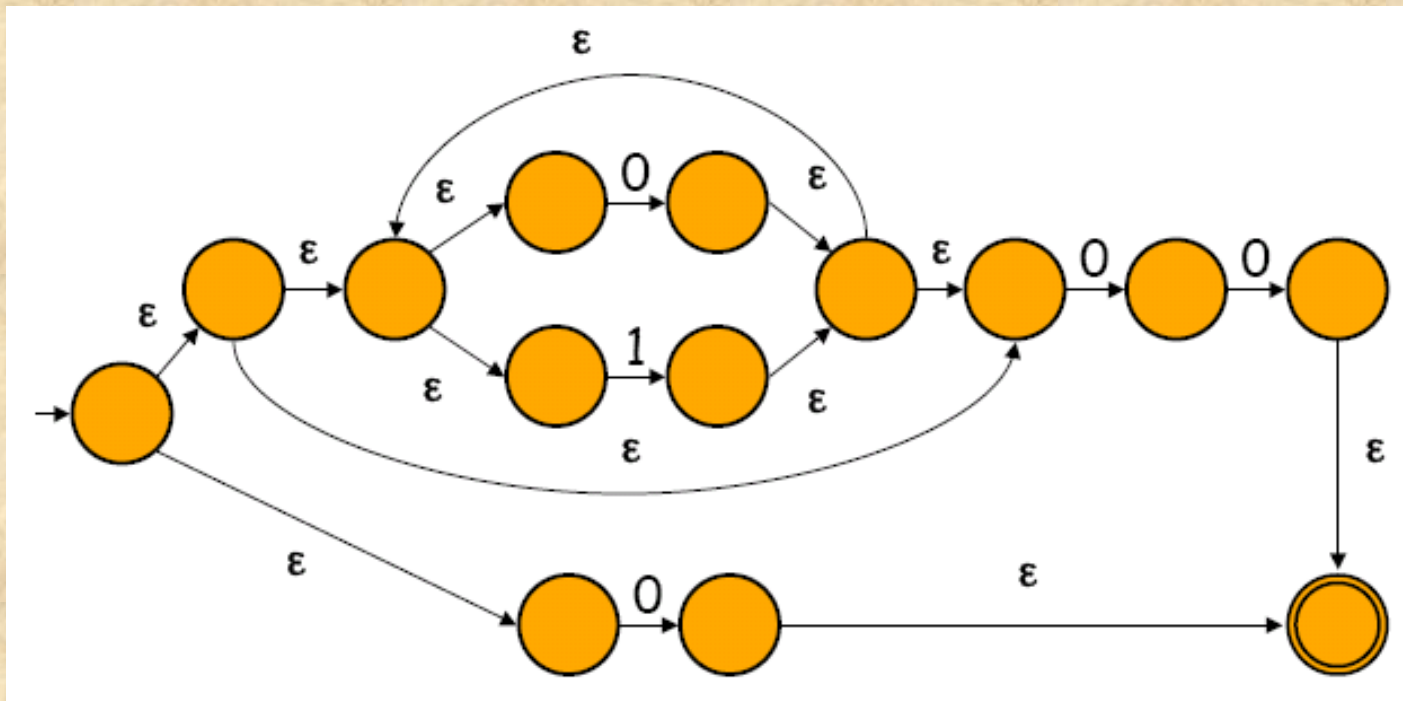
Εδώ ενώσαμε την 7 με την 7' αντί να βάλουμε $7 \rightarrow 7'$ με ϵ

Μη αιτιοκρατικά αυτόματα (7/7)

Παράδειγμα. Κατασκευή NFA για την έκφραση $(a|b)^*abb$



Ένθετο – παράδειγμα $RE \rightarrow NFA$



Παράδειγμα. Κατασκευή NFA την έκφραση $((0|1)^*00)|0$